Adam Lee
Dr. Levent Yilmaz
COMP3270
2/18/2026

HW2

Problem 1: Proof by Loop Invariant – arrayFind

Loop Invariant:
At the start of each iteration of the while loop, x is not contained in A[0..i−1].

Initialization:
Before the first iteration, i = 0. The subarray A[0..−1] is empty. Thus the invariant holds vacuously.

Maintenance:
Assume the invariant holds at the start of iteration i. If x == A[i], the algorithm correctly returns i. If not, i is incremented. Since A[i] ≠ x and x was not in A[0..i−1], x is not in A[0..i]. Thus the invariant is maintained.

Termination:
The loop stops when i = n. The invariant implies x is not in A[0..n−1]. Therefore returning −1 is correct.

Conclusion: The algorithm is correct.

Problem 2: Ordering by Big-Oh

Ordered from smallest to largest:

1/n (smallest)
2^100
log log n
sqrt(log n)
log^2 n
n^0.01
sqrt(n), 3n^0.5
5n, 2^(log n)
6n log n, n log_4 n
n log^2 n
n^(3/2), 4n^(3/2)
n^2, 4^(log n)
n^2 log n
n^3

2^n
4^n
2^(2n) (largest)

Problem 3: Min and Max Using Fewer than 3n/2 Comparisons

Recursive Strategy:
Divide array into two halves. Recursively compute min and max of each half. Combine results using two comparisons.

Recurrence:
$T(n) = 2T(n/2) + 2$
$T(1)=0$, $T(2)=1$

Using recursion tree, total comparisons:
$T(n) = (3n/2) - 2$

Thus fewer than 3n/2 comparisons.
Complexity: $\Theta(n)$.

Problem 4: Mystery Algorithm

(a) The algorithm computes the minimum element of the array.

(b) Recurrence:
$T(1) = c$
$T(n) = 2T(n/2) + c$

(c) Recursion Tree Table:

Level	# of Subproblems	Size of Each	Work per Node	Total Work Level
0 (Root)	1	n	c	c
1	2	n/2	c	2c
2	4	n/4	c	4c
...	2^k	n / 2^k	c	2^k c
log n (Leaves)	n	1	c	nc

Sum over all levels:
$T(n) = c(1 + 2 + 4 + ... + n)$

= 2cn − c
(d) Complexity: Θ(n)

Problem 5: T(n)=7T(n/8)+cn

(Recursion Tree Table)

Level	# of Subproblems	Size of Each	Work per Node	Total Work Level
0 (Root)	1	n	cn	cn
1	7	n/8	c(n/8)	7c(n/8)
2	7^2	n/8^2	c(n/8^2)	7^2 c(n/8^2)
...	7^k	n/8^k	c(n/8^k)	7^k c(n/8^k)
log_8 n (Leaves)	n^(log_8 7)	1	c	c n^(log_8 7)

Total work is:
T(n) = cn Σ (7/8)^k
= cn * 1/(1 − 7/8)
= 8cn − lower order term

Therefore, T(n) = Θ(n).

<u>Problem 6: Substitution Method</u>

T(n)=3T(n/3)+5,

Guess: T(n)=5n
Therefore prove: T(n)=Θ(n).
Base case:
T(1)= 5
5n=5(1)=5
Both equal so base case holds
Inductive hypothesis:
Assume that for all k < n, T(k) = 5k
Inductive step:
T(n)=3T(n/3)+5
T(n)=3(5(n/3))+5
= 5n+5
=5n+5
T(n) = 5(n) + 5, T(n)=Θ(n).

Inductive step confirms bound.
Constant c = 5.

Problem 7: Master Method

1) T(n)=2T(99n/100)+100n → Θ(n^{log_{100/99}2})
log100/992≈69
n^logba≈n^69
T(n)=Θ(n^69)

2) T(n)=16T(n/2)+n^3 log n → Θ(n^4)
a=16,b=2: n^log_2^16=n^4
n^3 logn < n^4
T(n)=Θ(n^4)

3) T(n)=16T(n/4)+n^2 → Θ(n^2 log n)
a=16,b=4: nlog_4^16=n^2
f(n)=Θ(n2)
T(n)=Θ(n^2 logn)

Problem 8: T(n)=2T(n−1)+1

Expand three times:

$$\begin{aligned} T(n) &= 2T(n-1)+1 \\ &= 2(2T(n-2)+1)+1 \\ &= 2^2T(n-2)+2+1 \\ &= 2^3T(n-3)+2^2+2+1 \end{aligned}$$

Pattern:

$$T(n) = 2^kT(n-k) + (2^{k-1}+\cdots+1)$$

Let k = n:

$$T(n) = 2^nT(0) + (2^{n-1}+\cdots+1)$$

Since $T(0) = 1$*:*

$$T(n) = 2^n + \sum_{i=0}^{n-1} 2^i$$

Using geometric series:

$$\sum_{i=0}^{n-1} 2^i = 2^n - 1$$

$$T(n) = 2^n + (2^n - 1)$$

Verification

$$2T(n-1) + 1$$
$$= 2(2^n - 1) + 1$$
$$= 2^{n+1} - 1$$

LHS = RHS

Complexity

$T(n)=\Theta(2n)$

Forward Substitution

$T(0) = 1$
$T(1) = 3$
$T(2) = 7$
$T(3) = 15$

Pattern:

$$= 2^{n+1} - 1$$